SEUPD@CLEF: Team NEXA

Paul Arlot[1], Andrea Di Tillo[1], Gaute Greiff Flagstad[1], Bita Khashechian[1], Danil Smirnov[1] and Marco Tomaiuoli[1]

[1] *University of Padua, Italy*

Abstract

This paper presents the system developed by Team NEXA for Task 1 of the CLEF 2026 CheckThat! Lab, focusing on the retrieval of scientific sources for claims found on social media. Given the challenge of linking informal web discourse to formal scientific literature across English, German, and French, we propose a multi-stage retrieval framework. Our approach combines traditional lexical search using BM25 with a neural re-ranking component based on multilingual transformer models. The system aims to bridge the linguistic and stylistic gap between social media posts and academic abstracts, providing a scalable solution to assist fact-checkers in verifying scientific information.

Keywords

Information Retrieval, Search Engine, CLEF, CheckThat!, Scientific Claim Verification, Lucene, Multilingual Search

1. Introduction

Information Retrieval systems play a fundamental role in enabling access to relevant data across massive, often heterogeneous corpora. In the context of the CheckThat! Lab at CLEF, which focuses on identifying the original scientific research behind social media claims, we have developed a configurable IR system. The motivation for this project comes from the need to fight disinformation by making fact-checking faster. As noted by CheckThat!, manually searching for scientific evidence is very slow for human experts. By automating this source retrieval process, we can help fact-checkers quickly link a claim to its original paper. A primary difficulty in this task is cross-language retrieval: social media posts formulated in English, French and German must be mapped to a corpus of scientific publications that is mainly in English, alongside a minority of other languages. To achieve this, we translated every post and document in English in order to be able to use the same approach regardless of the source language. Our system supports a wide range of preprocessing and indexing strategies, enabling us to experiment with different setups to address the vocabulary gap between informal web text and formal scientific abstracts.

The paper is organized as follows: Section 2 describes our approach; Section 3 explains our experimental setup; Section 4 discusses our main findings; finally, Section 5 draws some conclusions and outlooks for future work.

2. Methodology

In this section, we describe the architecture and components of our system. Our approach follows a multi-stage Information Retrieval pipeline combining traditional lexical matching with semantic representations.

"Search Engines", course at the master degree in "Computer Engineering", Department of Information Engineering, and at the master degree in "Data Science", Department of Mathematics "Tullio Levi-Civita", University of Padua, Italy. Academic Year 2025/2026

EMAIL: paullouisjean.arlot@studenti.unipd.it (P. Arlot); andrea.ditillo@studenti.unipd.it (A. D. Tillo); gaute.greiff.flaegstad@studenti.unipd.it (G. G. Flagstad); bita.khashechian@studenti.unipd.it (B. Khashechian); danil.smirnov@studenti.unipd.it (D. Smirnov); marco.tomaiuoli@studenti.unipd.it (M. Tomaiuoli)

2.1. System Configurability

2.2. Data Parsing

The first stage of the pipeline turns the two textual inputs of the system, the corpus of scientific publications and the set of input claims, into structured Java objects that the rest of the pipeline can work with. Both inputs are provided as JSON and share the same text cleaning routine, so we describe them together. On the data side we have two simple classes: `Publication` for documents and `Claim` for queries.

2.2.1. Documents (Publications)

Documents are read through an abstract `CommonParser` class that defines the iterator interface and exposes the shared text cleaning routine, with concrete subclasses for each input format. Since the corpus is provided as JSON, the only subclass we currently use is `JsonParser`.

`CommonParser` implements `Iterator<Publication>`, so it produces one `Publication` at a time as the input is read. Each `Publication` gets the fields we use for retrieval: a numerical identifier (`pubkey`), the `title`, `abstract`, `venue` and `authors`. Missing or `null` fields are replaced with empty defaults, and empty abstracts are filled with a placeholder token so that every document still produces a valid entry in the index.

`JsonParser` uses the Jackson library to read the corpus, which is stored as a JSON array of document objects. We process the stream incrementally rather than loading the whole file at once, which keeps memory usage flat regardless of corpus size. For each publication we map the JSON fields onto a `Publication` instance and run the abstract through the cleaning routine before it leaves the parser.

2.2.2. Queries (Claims)

Queries are loaded from a separate JSON topics file containing a list of claim objects. The file is small and only read once at the start of a run, so we parse it into a `List<Claim>` using a Jackson `ObjectMapper`, rather than going through the streaming parser used for documents. Each `Claim` keeps all three fields: the claim `index` (used as the query identifier in the TREC run file), the claim `text`, and the gold `pubkey` when present (used only at evaluation time).

To keep documents and queries aligned, the cleaning routine described next is applied to the claim text from inside the setter of the `Claim` class. By the time the searcher iterates over the list, both sides of the retrieval problem have gone through exactly the same preprocessing, which avoids the kind of subtle mismatches that arise when documents and queries are normalised differently.

2.2.3. Text Cleaning

Before any text reaches the analyzer, we pass it through a cleaning function (`cleanText`) that strips out the noise produced by the mix of scientific writing and social media content in the corpus. The function applies the following steps in order:

- **Unicode normalisation:** the text is converted to NFC form so the character representation is consistent.
- **HTML unescaping and stripping:** HTML entities are decoded and any leftover tags are removed.
- **URL removal:** hyperlinks are dropped so they do not end up as tokens.
- **Citation removal:** bracketed references (e.g. "[1, 2–5]") and author–year citations (e.g. "(Rossi et al., 2023)") are removed.
- **Section header removal:** structural labels that appear in scientific abstracts, such as *Abstract*, *Methods*, *Results* or *Conclusions*, are stripped.

- **Symbol and emoji removal:** mathematical and scientific symbols (e.g. °, ±, ≥) and emoji characters are replaced with spaces.
- **Social media artefacts:** Twitter-style mentions (e.g. `@user`) are removed.
- **Whitespace collapsing:** runs of whitespace are collapsed into a single space and any leading or trailing whitespace is trimmed.

This step is important because the corpus mixes formal abstracts with informal claims. Without it the analyzer would have to deal with a lot of noisy tokens that could decrease matching quality.

2.3. Query Expansion

To merge the vocabulary between informal social media claims and formal scientific abstracts, the system supports three query expansion strategies that can be turned on independently from the configuration file. Each one adds extra terms to the Boolean query produced by the searcher, but with different sources and different boost weights so that the original claim terms always weigh more than the expansions.

2.3.1. Static Dictionary Expansion

The simplest strategy is a manually curated synonym dictionary, loaded from the file referenced by the `synonymsFile` parameter. During query construction (`buildLexicalQuery`), each token of the claim is looked up in the dictionary, and any matching synonyms (for example, "car" → "automobile", "vehicle") are added to the abstract subquery with a boost of `0.5f`, half the weight of the original term. This keeps the original tokens dominant in the ranking while still letting the expansion contribute. Because the dictionary lives on disk next to the index, the overhead is negligible, but the coverage is limited to whatever entries the file contains.

2.3.2. LLM-based Expansion

To capture relations beyond a fixed dictionary, the system can expand the query using a Large Language Model. The claim text is sent to the Groq API with the configured `openApiKey` and `llmModel` (currently `llama-3.1-8b-instant`). The model returns a short list of related terms, for example "CO_2", "pollution" and "glaciers" for "climate change". These terms are added to the title and abstract subqueries with a boost of `0.4f`.

This route is more expressive than the static dictionary, but it adds a network round trip to every query, and the free tier of the Groq API is rate-limited to roughly 30 requests per minute, which makes it impractical for batch experiments without throttling.

2.3.3. Pseudo-Relevance Feedback

The third strategy is Pseudo-Relevance Feedback (PRF), which expands the query using the local index instead of an external resource. PRF works on the assumption that the top-ranked documents returned by a first search are likely to be relevant, and reuses their content as a source of related terms. The procedure has two stages:

1. **Initial search.** The system runs the original query and keeps the top-k documents (we use $k = 3$).
2. **Expansion and re-search.** The title and abstract of each of those documents are tokenised, numeric and very short tokens are dropped, and the five most frequent remaining terms are appended to the original query with a boost of `0.3f`. The search is then re-executed against the index.

The appeal of PRF is that the expansion terms come straight from the corpus, so they are guaranteed to be in-vocabulary for the index. The cost is that every claim now requires two Lucene searches instead of one, which roughly doubles retrieval time. For this reason PRF is disabled in the timed runs we report.

2.3.4. Earlier Gemini-based Expansion

An earlier version of the LLM expansion module relied on Google's Gemini API rather than Groq, with a different integration shape. A small Python service ran alongside the Java search engine and acted as a bridge between the two. For each query, the service first looked up the claim text in a local on-disk cache; if the same query had been expanded before, the cached terms were returned immediately. Otherwise the raw claim was forwarded to the Gemini API with a prompt asking the model to ignore emojis and opinions, list related synonyms, and return between 10 and 15 keywords. The response was then written back to the cache and returned to the Java side.

This approach was not carried into the final pipeline. It required running a separate Python service alongside the Java search engine, with its own dependency stack, cache file, and HTTP coupling point between the two processes. The Groq-based path described above covers the same use case (LLM-generated synonyms with on-disk caching) directly from Java, with one fewer moving part. The Gemini API was also less reliable in our setup, which made the trade-off easier.

2.4. Text Analysis

The text analysis phase transforms raw textual input into a stream of tokens that can be indexed and later retrieved. Initially, we tried to use language-specific analyzer classes but using a unique English Analyser after translation was simpler and offered better results. The analysis is performed through an analyzer classes, which extend the Apache Lucene `Analyzer` framework.

The analyzer follows an architecture composed of three main components:

- **Tokenizer:** Responsible for splitting the input text into raw tokens.
- **Token Filters:** Sequentially modify, normalize, or enrich the token stream.
- **Stemming/Lemmatization:** Reduces tokens to their base or root forms to improve generalization.

The overall pipeline is dynamically configured through external YAML configuration files, allowing flexible adaptation of processing strategies and experimental settings.

2.4.1. Tokenizer Configurations

Multiple tokenization strategies are supported, enabling the system to handle different types of input text:

- **Standard Tokenizer:** Uses Lucene's rule-based tokenizer to handle punctuation, digits, and word boundaries.
- **Letter Tokenizer:** Emits tokens composed only of alphabetic characters, treating non-letter characters as delimiters.
- **Whitespace Tokenizer:** Splits input strictly on whitespace characters without processing punctuation.
- **NLP Tokenizer:** Uses Apache OpenNLP models to perform linguistically informed tokenization.

The tokenizer type is selected through configuration files, allowing to try different strategies.

2.4.2. Token Filters

Token filters are applied after tokenization to normalize and enrich the token stream. These filters are categorized into general (language-independent) and language-specific filters.

General Token Filters These filters perform general normalization operations across all languages:

- **LowerCaseFilter:** Converts all tokens to lowercase.
- **ICUFoldingFilter:** Normalizes accented and special characters into ASCII equivalents.
- **NBSPFilter:** Removes non-breaking spaces and trims tokens.
- **RemoveDuplicatesTokenFilter:** Eliminates duplicate tokens.
- **LengthFilter:** Removes tokens that are too short or too long.
- **RepeatedLetterFilter:** Normalizes tokens with repeated characters (e.g., "soooo" → "soo").

Enrichment Filters To improve semantic representation, additional filters are applied:

- **AbbreviationExpansionFilter:** Expands abbreviations using predefined mappings (e.g., "ml" → "machine learning").
- **CompoundPOSTokenFilter:** Uses Part-of-Speech tagging to combine semantically related tokens into compound expressions.
- **ShingleFilter:** Generates n-grams to capture local contextual information.
- **PositionFilter:** Normalizes positional increments for indexing consistency.

Language-Specific Filters Each analyzer applies additional filters tailored to linguistic characteristics:

- English-specific filters (e.g., possessive removal, English stopwords)
- French-specific filters (e.g., elision handling, French stopwords)
- German-specific filters (e.g., compound handling and normalization)

This design ensures that each language is processed according to its grammatical and lexical properties. Since we translate everything to English, we are using only English-specific filters.

2.4.3. Stemming and Lemmatization

The system supports multiple strategies for reducing tokens to their canonical forms: **Porter Stemmer**, **Snowball Stemmer**, **OpenNLP Lemmatizer**, **No Stemming**.

The selected method is controlled through configuration files and can vary depending on the experimental setup.

2.5. Document Indexing

The document indexing stage transforms processed publications into a searchable inverted index using Apache Lucene. This structure enables efficient retrieval of relevant documents during the query phase. In our system, indexing is handled by the `DirectoryIndexer` component, which coordinates parsing, preprocessing, and storage of documents.

2.5.1. Indexing Workflow

The indexing workflow operates over a collection of JSON files containing scientific publications. Each file is parsed to extract individual documents, represented internally as structured `Publication` objects. Only documents containing meaningful textual content (such as title or abstract) are considered for indexing.

In addition to standard preprocessing, the system supports optional enhancements. When enabled, documents written in languages different from the target language can be translated prior to indexing. Moreover, the system can generate dense vector representations of the textual content by interacting with an external embedding service. These embeddings are stored alongside the indexed documents to support future semantic retrieval extensions.

Each processed publication is converted into a Lucene `Document` and added to the index using an `IndexWriter`. During this process, statistics such as the number of indexed documents and total indexing time are recorded.

2.5.2. Field Representation

Document content is stored in structured fields, such as title and abstract. A custom field type is defined to support advanced indexing features, including term frequencies, positional information, and character offsets.

The system also enables storage of term vectors, which allows more advanced retrieval functionalities such as phrase matching, proximity queries, and potential re-ranking strategies. Additionally, the original textual content can be stored in the index to support downstream processing steps.

2.5.3. Processing Pipeline

The indexing procedure follows a well-defined sequence of steps. First, the system initializes the index writer with the configured analyzer and output directory. Then, for each document, the following operations are performed:

- Parsing the input file into structured publication objects
- Filtering out incomplete or invalid entries
- Detecting the language of the document
- Applying optional translation if required
- Generating semantic embeddings (if enabled)
- Converting the document into a Lucene-compatible format
- Adding the document to the index

Once all documents have been processed, the index is finalized by committing changes and closing the writer.

This modular and extensible design allows the indexing system to integrate multiple preprocessing strategies, multilingual handling, and semantic enrichment, while keeping the core indexing logic independent and reusable.

2.6. Retrieval Phase

The retrieval phase is responsible for matching input claims with relevant scientific publications stored in the indexed collection. This component is implemented through the `Searcher` class, which performs query processing, executes search operations over the Lucene index, and outputs ranked results.

The system initializes a Lucene `IndexSearcher` over the indexed data and employs the BM25 similarity function for ranking. Queries are provided as a collection of claims stored in JSON format, where each claim represents a short textual statement that must be linked to relevant documents.

2.6.1. Query Formulation

Each claim is processed independently. The textual content is first transformed into a structured query using a `QueryBuilder`, which applies the same analyzer used during indexing. This ensures that both documents and queries undergo consistent preprocessing, reducing mismatches due to tokenization or normalization.

The resulting terms are used to construct field-specific queries targeting different parts of the indexed documents.

2.6.2. Structured Boolean Queries

Instead of relying on a single-field representation, the system generates separate subqueries for the title and abstract fields of each document.

These subqueries are combined using a Boolean query with `SHOULD` clauses, allowing documents to be retrieved if they match either field. This formulation increases recall while maintaining flexibility in matching.

To further refine the ranking, different importance weights are assigned to each field through boosting. In particular, matches occurring in the title field are given higher importance than those in the abstract. This results in a weighted query structure that prioritizes concise and informative matches while still considering broader contextual information.

2.6.3. Ranking Mechanism

Once the Boolean query is constructed, it is executed using the Lucene search engine. The system retrieves the top-k documents according to their BM25 scores. This ranking function considers the frequency of query terms, their rarity across the collection, and document length normalization.

The number of retrieved documents is configurable and can be adjusted depending on the evaluation setup.

2.6.4. Result Formatting

The retrieved results are written to an output file following the TREC evaluation format. Each entry includes the query identifier, document identifier, rank position, score, and run identifier. This standardized format ensures compatibility with external evaluation tools commonly used in information retrieval benchmarks.

3. Experimental Setup

3.1. Used Collections

We used the datasets provided by CheckThat! on https://huggingface.co/datasets/sschellhammer/CT26_Task1_SourceRetrievalForScientificWebClaims
It consist of 10 json files:

- collection_data.json - 10000 documents
- en_dev.json - 3905 posts
- fr_dev.json - 702 posts
- de_dev.json - 386 posts
- en_train.json - 14977 posts
- fr_train.json - 2807 posts
- de_train.json - 1460 posts
- final_en_test.json - 6076 posts

- final_fr_test.json - 1220 posts
- final_de_test.json - 876 posts

3.2. Evaluation Measure

The main evaluation measure used is the Mean Reciprocal Rank at 5 (MRR@5):

$$MRR@5 = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \begin{cases} \frac{1}{rank_i} & \text{if } rank_i \leq 5 \\ 0 & \text{if } rank_i > 5 \end{cases}$$

With:

- $|Q|$: The total number of queries being evaluated.
- i: The index of the current query (from 1 to $|Q|$).
- $rank_i$: The rank position of the *first* relevant result for query i.
  - If the first relevant result is at rank 1, 2, 3, 4, or 5, its score is $\frac{1}{rank_i}$.
  - If the first relevant result is at rank 6 or below (or if there are no relevant results), its score is 0.

3.3. Git repository and its organization

The entire project, including the source code and some runs is available on Bitbucket at the following URL : https://bitbucket.org/upd-dei-stud-prj/seupd2526-nexa/
The repository is structured as follows:

- `code`: the source code of the project
- `homework-1`: initial report at the end of the evaluation phase
- `runs`: some runs in trec_eval format generated by our IR system

3.4. Hardware used for experiments

- CPU:
- RAM:
- GPU:
- Storage:

4. Results and Discussion

In this section we report the experiments that led to the final lexical and hybrid configurations of the system. Because the search space is much larger than what we could grid-search, we tuned the components in order, fixing the winner of each phase before moving on:

1. Section 4.1 fixes the analyzer pipeline (tokenizer, stemmer, stop list, filters).
2. Section 4.2 fixes the retrieval model and BM25 parameters.
3. Section 4.3 fixes the fielded query (boosts, query mode, IDF pruning).
4. Section 4.4 reports two expansion strategies (WordNet, PRF) that did not work.
5. Section 4.5 ablates the components of the final lexical system.
6. Section 4.6 reports the dense fusion and reranking results.
7. Section 4.7 compares the runs we submitted.

All scores are MRR@5 on the dev split, reported per language (EN, FR, DE) with the average across languages. Hybrid and reranking experiments operate on the top-50 candidates of the lexical run. The dataset, dev splits, hardware, and evaluation tooling are described in Section 3.

4.1. Lexical Analyzer Tuning

We start by fixing the analyzer pipeline that the rest of the chapter builds on. Table 1 varies the tokenizer (StandardTokenizer vs. WhitespaceTokenizer) and the stemmer (KStem, Snowball, Porter, EnglishMinimal, or none), with the English stop list either active or disabled. All other components (lowercasing, ICU folding, the EnglishPossessiveFilter) are held constant, so each row isolates one change. Word shingles are reported separately below.

Table 1
Tokenizer and stemmer comparison on the EN/FR/DE dev sets, MRR@5. Variants share the standard pipeline (lowercase, ICU folding, English stop list) and differ only in the stemmer or tokenizer.

Configuration	EN	FR	DE	Avg
Standard, no stop, no stem	0.4826	0.4453	0.3014	0.4098
Standard, stop, no stem	0.4864	0.4674	0.3225	0.4254
Standard, stop, EnglishMinimal	0.4894	0.4817	0.3304	0.4338
Standard, stop, Snowball	0.4869	0.4785	0.3358	0.4337
Standard, stop, Porter	0.4867	0.4801	0.3381	0.4350
Standard, stop, KStem	**0.4883**	**0.4901**	**0.3354**	**0.4379**
Whitespace, stop, KStem	0.4422	0.4087	0.2746	0.3752

The tokenizer dominates every other choice. Switching from the StandardTokenizer to the WhitespaceTokenizer costs about six points of average MRR@5 (0.4379 vs. 0.3752) in all three languages, because the WhitespaceTokenizer keeps punctuation attached to tokens and does not split hyphenated compounds such as "COVID-19" or "SARS-CoV-2". The Standard pipeline also handles FR and DE without language-specific rules; we tried language-specific synonym and POS filters on those two languages and disabled them, since they added noise without measurable gain.

Within the Standard rows, the stop list matters more than the choice of stemmer. Removing it drops the average by about 1.6 points, and by 2.3 points on FR, where common function words match aggressively against translated abstracts. The four stemmers, by contrast, fall within 0.005 of each other on the average. KStem is narrowly best and the most conservative of the four: it conflates *vaccination*/*vaccinate* but leaves *corona*/*coronary* distinct, which matters in a domain where over-stemming can reduce retrieval accuracy. We therefore lock in the Standard tokenizer, the English stop list, and KStem for the rest of the chapter.

We also tested word shingles on top of this pipeline. Two-shingles averaged 0.3855 and three-shingles 0.4040, both well below the 0.4379 unigram baseline. Indexing unigrams plus bigrams reduces BM25's IDF weight on the precise scientific terms in the original sentence, and since claims are already complete sentences, shingles add no phrase signal that unigram BM25 was missing. Shingles are disabled in all later experiments. The EnglishPossessiveFilter, which strips trailing *'s*, changes the average by less than 0.001, so we keep it on at no runtime cost.

4.2. Retrieval Model and BM25 Parameters

With the analyzer fixed, we select and optimize a retrieval ranking function. Table 2 evaluates BM25 against competing approaches: the Classic TF-IDF model, LM Dirichlet (with $\mu \in \{500, 1000\}$), and LM Jelinek-Mercer (with λ=0.5). We then systematically test BM25's hyperparameters (k_1 and b) to verify that Lucene's defaults (k_1=1.2, b=0.75) are optimal.

BM25 wins on the average by about 1.2 points over the strongest language-model variant and by 7.6 points over Classic TF-IDF. Classic is the worst row because it has no document-length normalisation, so long abstracts accumulate score regardless of how concentrated their relevant terms are. LM Dirichlet degrades as μ grows from 500 to 1000, since a stronger prior pulls scores towards the collection background and washes out the highly specific vocabulary that

Table 2
Retrieval model comparison, MRR@5 on the dev sets. BM25 uses Lucene defaults (k_1=1.2, b=0.75); only the most informative LM Dirichlet and LM Jelinek-Mercer rows are shown.

Model	EN	FR	DE	Avg
BM25 (k_1=1.2, b=0.75)	**0.4838**	**0.4887**	**0.3342**	**0.4356**
Classic (TF-IDF)	0.4130	0.3948	0.2715	0.3597
LM Dirichlet (μ=500)	0.4683	0.4652	0.3357	0.4231
LM Dirichlet (μ=1000)	0.4526	0.4434	0.3098	0.4019
LM Jelinek-Mercer (λ=0.5)	0.4784	0.4706	0.3220	0.4236

distinguishes scientific abstracts. LM Jelinek-Mercer is closer but never matches BM25 in any language.

We then swept $k_1 \in \{0.5, 0.8, 1.0, 1.2, 1.5, 2.0, 3.0\}$ against $b \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$. The b parameter dominates: moving from b=0 to b=0.75 adds about five points of average MRR@5 at every k_1, whereas at b=0.75 the entire k_1 column varies by less than 0.001. The optimum sits at k_1=1.0, b=0.75 with an average of 0.4357, within 0.0015 of the Lucene defaults. We lock in k_1=1.0, b=0.75 for the rest of the chapter.

4.3. Fielded Search and Query Construction

We now tune how the query is assembled: which fields to boost, whether terms are required or optional, and how aggressively to prune the query. Table 3 sweeps the title, abstract, and authors boosts in three parts (title with abstract fixed, abstract with title fixed, then authors on top of the winning ratio). Table 4 then prunes the query to the top-K tokens by IDF. The strictness of the Boolean query (SHOULD vs. MUST) is reported in prose, since the effect is too one-sided to need a table.

Table 3
Field boost sweeps on the EN/FR/DE dev sets, MRR@5. Part A varies the title boost with the abstract boost fixed at 1.0; Part B varies the abstract boost with the title boost fixed at 2.0; Part C adds the authors boost on top of the winning combination (title boost 2, abstract boost 3).

Boost weight	EN	FR	DE	Avg
Part A: title boost (abstract boost = 1.0)				
1.0	0.4997	0.5026	0.3577	0.4533
1.5	0.4929	0.4964	0.3475	0.4456
2.0	0.4838	0.4882	0.3352	0.4357
3.0	0.4635	0.4567	0.3159	0.4121
Part B: abstract boost (title boost = 2.0)				
1.0	0.4838	0.4882	0.3352	0.4357
1.5	0.4956	0.5000	0.3506	0.4487
2.0	0.4997	0.5026	0.3577	0.4533
3.0	0.4990	0.5066	0.3604	0.4553
Part C: authors boost (title boost = 2.0, abstract boost = 3.0, topK=30)				
0.5	0.4888	0.4938	0.3373	0.4400
1.0	0.4990	0.5066	0.3604	0.4693
1.5	**0.5172**	**0.5145**	**0.3841**	**0.4719**
2.0	0.5160	0.5130	0.3820	0.4715
3.0	0.5050	0.5020	0.3754	0.4608

Parts A and B fix the title-to-abstract boost ratio. With the abstract boost held at 1.0, raising the title boost above 1.0 only hurts; with the title boost held at 2.0, raising the abstract boost up to 3.0 monotonically improves. The winning combination is a title boost of 2 and an

abstract boost of 3, and what matters is the 2:3 ratio rather than the absolute values. Claims are tweet-style paraphrases of findings, not of titles, so the abstract is the primary evidence field, while the title still contributes a smaller but non-zero signal.

Part C adds the authors field on top of these settings, with the peak at an authors boost of 1.5 (0.4719 average), about 1.5 points above the no-authors row. Pushing higher hurts as mismatched proper nouns start to dominate. The DE column gains the most (about 2.4 points), consistent with German-translated tweets more often naming researchers than the English originals. We also swept a venue boost from 0.0 to 0.5 with no measurable effect.

We then varied the strictness of the Boolean query, marking the top-IDF terms as MUST and the rest as SHOULD. Even the lightest mixed variant (must= 2) drops the average to 0.087, and a full MUST query collapses to 0.007. Claims paraphrase findings rather than quoting them, so any hard term constraint reduces recall heavily. We use pure SHOULD throughout the rest of the chapter.

Table 4
IDF-based query pruning on top of the fielded query, MRR@5. Each claim is tokenised with the English analyzer; only the top-K tokens by IDF are passed as the query.

K	EN	FR	DE	Avg
all (no pruning)	0.4838	0.4882	0.3352	0.4357
10	0.4115	0.4189	0.2887	0.3730
15	0.4696	0.4658	0.3182	0.4179
20	0.4914	0.4798	0.3379	0.4364
30	**0.4956**	**0.4860**	**0.3395**	**0.4404**
40	0.4956	0.4860	0.3395	0.4404
50	0.4956	0.4860	0.3395	0.4404

For the IDF pruning sweep, the score climbs from K=10 to K=30 and then flattens: K=30, 40, and 50 are identical, since most claims contain fewer than 30 distinct content tokens after stop-word filtering. Smaller values are clearly harmful: at K=15 the average loses 2.3 points and at K=10 over six points, because genuine scientific terms get dropped along with ordinary content words. We set K=30. The locked-in lexical configuration is therefore BM25 (k_1=1.0, b=0.75) with field boosts (title 2, abstract 3, authors 1.5), SHOULD mode, and the query pruned to the top 30 IDF tokens.

4.4. Failed Expansion Approaches

The principal challenge in this task is a vocabulary gap: claims are informal, tweet-style paraphrases of scientific findings, while the indexed documents are formal abstracts. Two standard remedies are query expansion by synonym injection and pseudo-relevance feedback (PRF), which expands the query with terms drawn from the top retrieved documents. We tested both on top of the analyzer fixed in Section 4.1, expecting them to bridge the informal-to-formal gap. Both hurt retrieval monotonically as expansion grows.

WordNet expansion degrades the score monotonically: adding at most one synonym per word costs 3.9 points of average MRR@5, and at most five synonyms costs 9.1 points, leaving the system far below its unexpanded baseline. Three compounding factors explain this. First, WordNet is a general-vocabulary lexicon and covers very few of the precise scientific terms that appear in the corpus ("efficacy", "mRNA", "SARS-CoV-2") so synonyms are generated mainly for the common words in the claim, not the discriminative ones. Second, the general synonyms that WordNet does produce introduce ambiguity: "system" in *immune system* receives synonyms that point to entirely unrelated everyday senses, introducing tokens that match irrelevant documents. Third, every injected synonym increases query length, which reduces BM25's IDF weighting on the original precise terms via length normalisation, making the genuine discriminative signal

Table 5
Query expansion experiments, MRR@5. All runs use the analyzer configuration fixed in Section 4.1.

Configuration	EN	FR	DE	Avg
Lexical baseline (no expansion)	0.4883	0.4901	0.3354	0.4379
WordNet, max 1 syn/word	0.4582	0.4352	0.3040	0.3991
WordNet, max 3 syn/word	0.4262	0.3993	0.2829	0.3695
WordNet, max 5 syn/word	0.4047	0.3759	0.2591	0.3466
PRF (3 docs, 3 terms)	0.4825	0.4763	0.3351	0.4313
PRF (5 docs, 3 terms)	0.4829	0.4790	0.3310	0.4310
PRF (10 docs, 10 terms)	0.4776	0.4692	0.3317	0.4262

weaker rather than stronger.

PRF avoids the dictionary mismatch problem by extracting expansion terms from documents the system has already retrieved. The degradation is smaller (0.7–1.2 points), but the direction is consistently negative. The root cause is topic drift: in a corpus dominated by COVID and vaccine research, the top retrieved documents for any health claim are neighbouring papers on similar but distinct subtopics. Feeding their most frequent terms back into the query shifts the focus toward the broader topic cluster and away from the specific claim. BM25 length normalisation penalises the expanded query for the same reason as with WordNet, compounding the drift.

Both strategies are disabled in all later experiments. The vocabulary gap they were meant to address is real, but adding tokens to the lexical query is the wrong tool. Section 4.6 shows that dense fusion with BGE-M3 does close the gap, because embeddings place tweet-style paraphrases and formal abstracts close in a shared continuous space without any additional tokens entering the BM25 scoring path.

4.5. Ablation of the Lexical System

To isolate the contribution of each lexical component, we ablate them one at a time on top of the full best configuration locked in across Sections 4.1 to 4.3. Table 6 reports each ablation as a one-component change against that reference, plus a row that removes all three tunable components together, a row that adds the venue boost on top, and two rows that replace SHOULD with stricter Boolean modes. All runs share a single index, so differences come only from the component being changed.

Table 6
Ablation of the lexical system, MRR@5. The reference is the full configuration (BM25 k_1=1.0, b=0.75, field boosts title 2, abstract 3, authors 1.5, topK=30). Each row removes or changes exactly one component.

Configuration	EN	FR	DE	Avg	Δ Avg
Full best (reference)	0.5172	0.5145	0.3841	0.4719	n/a
− authorsBoost (set to 0)	0.5086	0.4973	0.3646	0.4568	−0.0151
− topKQueryTerms (use all terms)	0.5100	0.5131	0.3826	0.4686	−0.0034
− abstractBoost (set to 1)	0.4983	0.4885	0.3548	0.4472	−0.0247
− all three (simple baseline)	0.4874	0.4919	0.3323	0.4372	−0.0347
+ venueBoost = 0.5	0.5174	0.5155	0.3835	0.4721	+0.0002
queryMode = mixed (must= 2)	0.0752	0.1233	0.0635	0.0873	−0.3846
queryMode = must (all)	0.0151	0.0043	0.0026	0.0073	−0.4646

Ranked by impact, the abstract boost is the strongest single contributor: setting it back to 1 costs 2.5 points of average MRR@5. The authors boost is next at 1.5 points, and IDF pruning a smaller but still positive 0.3 points. Removing all three at once costs 3.5 points, slightly less

than the 4.3-point sum of the individual deltas; this sub-additivity is expected, since the three components partially recover the same correct documents rather than improving orthogonal aspects of the ranking. The venue boost lands at +0.0002 on the average and is not adopted. The authors field is also visibly more useful on DE than on EN (−2.0 points vs. −0.9), consistent with German-translated tweets naming researchers more often than the English originals.

The two queryMode rows reproduce the collapse already seen in Section 4.3: switching just the two highest-IDF terms to MUST drops the average to 0.087, and a fully MUST query is essentially zero in every language. No claim text appears verbatim in any abstract, so the system depends on every term being optional. This confirms SHOULD as the only viable Boolean mode and closes the lexical tuning.

4.6. Hybrid Retrieval: Lexical and Dense Fusion

On top of the lexical system fixed in the previous sections, we add a dense retrieval signal from a multilingual sentence encoder and combine the two scores. We compare two encoder families and two ways of combining the scores.

4.6.1. BGE-M3 Reranking and Fusion

BGE-M3 [**?**] is a multilingual sentence encoder that produces 1024-dimensional dense embeddings. For each query we encode the claim text and compare it against the pre-computed document embeddings by cosine similarity. We test two ways of incorporating this signal. In the reranking-only variant, the top-50 candidates from the lexical run are re-ordered by cosine similarity alone, with no BM25 score involved. In the fusion variant, the two scores are combined as

$$s = (1 - \alpha) \cdot \widetilde{s}_{\mathrm{BM25}} + \alpha \cdot \widetilde{s}_{\cos}, \tag{1}$$

where $\widetilde{s}_{\mathrm{BM25}}$ and $\widetilde{s}_{\cos}$ are the BM25 and cosine scores after min-max normalisation within each query's top-50 candidate set, and $\alpha \in [0, 1]$ controls the relative weight of the dense signal.

Table 7
Lexical baseline vs. BGE-M3 reranking and fusion, MRR@5. Reranking reorders the top 50 lexical hits by cosine; fusion combines $(1-\alpha) \cdot \mathrm{BM25}_{\mathrm{norm}} + \alpha \cdot$ cosine with both scores min-max normalised within each query's top-50.

Method	EN	FR	DE	Avg
Lexical baseline	0.5172	0.5145	0.3841	0.4719
BGE-M3 reranking only	0.5256	0.5280	0.3967	0.4834
BGE-M3 fusion (α=0.70)	**0.5768**	**0.5770**	**0.4331**	**0.5290**

We swept α from 0.50 to 0.90 in steps of 0.02. The per-language peaks fall in the narrow range $[0.69, 0.73]$ (EN and FR peak near 0.73, DE near 0.69), and the MRR@5 curve is flat within 0.002 of its peak for every language between α=0.67 and α=0.75. We therefore adopt α=0.70 as a single practical setting that is within 0.002 of every per-language optimum.

Reranking alone adds only 1.2 points of average MRR@5. The lexical run already places the correct document near the top of the 50-candidate list for many queries, leaving little room for re-ordering to help. Fusion gains 5.7 points, a much larger margin, because it can promote candidates that BM25 ranked anywhere in positions 6 to 50: a document whose precise vocabulary differs from the claim but whose meaning is close is scored low by BM25 and high by the cosine, and the combined score moves it into the top 5. The optimal α=0.70 assigns the embedding signal 2.3 times the weight of BM25, consistent with BM25 operating near its ceiling after the careful lexical tuning in the previous sections and the dense signal providing genuinely complementary evidence.

4.6.2. SPECTER2 vs BGE-M3

BGE-M3 is a general-purpose multilingual encoder. We also tested SPECTER2 [**?**], a domain-specific encoder trained on scientific paper-to-paper citation proximity, on the hypothesis that its scientific specialisation would outweigh the loss of multilingual coverage. The hypothesis does not hold.

Table 8
SPECTER2 vs. BGE-M3 at their respective best fusion weights. Both use the same 50-candidate input lists from the lexical run.

Method	EN	FR	DE	Avg
Lexical baseline	0.5172	0.5145	0.3841	0.4719
SPECTER2 reranking only	0.4451	0.4233	0.3377	0.4020
SPECTER2 fusion (α=0.60)	0.5524	0.5475	0.4150	0.5050
BGE-M3 fusion (α=0.70)	**0.5768**	**0.5770**	**0.4331**	**0.5290**

SPECTER2 reranking alone drops 7 points below the lexical baseline (0.4020 vs. 0.4719), while BGE-M3 reranking adds 1.2 points. The gap is explained by training objective and query distribution: SPECTER2 is trained on paper-to-paper citation proximity, so its representations capture which documents are co-cited rather than which documents answer a question. Its `adhoc_query` adapter was not exposed to informal social media text during training, so tweet-style claim embeddings land far from the abstract embeddings in its space. BGE-M3 is trained on a broad mixture of text pairs that includes informal-to-formal cross-domain matching, making it a natural fit for bridging claim paraphrases to scientific abstracts. The optimal fusion weight for SPECTER2 is α=0.60, lower than the 0.70 required for BGE-M3, indicating that the SPECTER2 signal is weaker and needs BM25 weighted more heavily to avoid degrading the result.

As a diagnostic, we also tried pairing SPECTER2 document embeddings with BGE-M3 query embeddings by aligning them into a common 768-dimensional space via PCA. The cross-space cosine collapses to an MRR@5 of approximately 0.025, confirming that the two embedding spaces are geometrically incompatible and cosine similarity is meaningless across them. When fused with BM25, the best alpha drops to 0.10 and the gain over the lexical baseline is between 0.0001 and 0.01 depending on the language, essentially zero. The failure is therefore not limited to SPECTER2's query-side representation: its document representations are also misaligned with what claim retrieval requires. BGE-M3 at α=0.70 is the encoder of choice.

4.7. Final Submitted Runs

Combining the winners of each phase above gives two configurations submitted to the official evaluation. Run 1 is the pure lexical system, included as a reproducible baseline that depends only on the Lucene index and requires no GPU at query time. Run 2 is the primary submission: the same lexical system with BGE-M3 score fusion at α=0.70, which is the best configuration identified across all experiments on the dev set.

Table 9
Submitted runs and dev-set MRR@5 per language.

Run	Description	EN	FR	DE	Avg
Run 1	Lexical only (best lexical config)	0.5172	0.5145	0.3841	0.4719
Run 2	Hybrid: lexical + BGE-M3 fusion (α=0.70)	0.5768	0.5770	0.4331	0.5290

Run 2 gains 5.7 points of average MRR@5 over Run 1 on the dev set. The improvement is consistent across all three languages: +6.0 points on EN, +6.2 on FR, and +4.9 on DE, so no language is disproportionately favoured or penalised by the fusion. All hyperparameters — field

boosts, k_1, b, α, and topK — were tuned exclusively on the dev split, so the test-set scores will be the first out-of-sample measurement. We report both runs so that the lexical contribution and the embedding gain can be read separately in the official results.

5. Conclusions and Future Work

Provide a summary of what are the main achievements and findings.

Discuss future work, e.g. what you may try next and/or how your approach could be further developed.

6. Group Members Contribution

Paul Arlot Initialized the Maven build environment and project architecture. Implemented a first iteration of the pipeline. Did some run on the cluster provided by the school. Contibuted to the report, particularly the experimental setup section.

John Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Jane Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

John Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Jane Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

John Doe Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.